

# Đồ thị dây cung và đồ thị khoảng

Ghi chú theo slide

Chen Danqi  
Đại học Thanh Hoa

*Nguồn gốc: Chordal Graph and Interval Graph*

Graph Theory / Chordal Graph and Interval Graph - Chen Danqi.pptx

## 1 Khái niệm cơ bản

Các slide bắt đầu bằng một số khái niệm chuẩn của lý thuyết đồ thị. Một đồ thị  $H = (V_H, E_H)$  là **đồ thị con** của  $G = (V, E)$  nếu  $V_H \subseteq V$  và  $E_H \subseteq E$ . Nếu  $E_H$  chứa đúng mọi cạnh của  $G$  có cả hai đầu trong  $V_H$ , thì  $H$  là **đồ thị con cảm sinh**.

Một **clique** là một tập đỉnh mà mọi cặp đỉnh trong tập đều kề nhau. Một clique là **cực đại** nếu nó không là tập con thật sự của clique khác. Một clique là **lớn nhất** nếu số đỉnh của nó là lớn nhất có thể; kích thước ấy được gọi là **số clique** và ký hiệu  $\omega(G)$ .

**Tô màu nhỏ nhất** là gán màu cho các đỉnh sao cho hai đỉnh kề nhau có màu khác nhau và số màu dùng là ít nhất; số màu tối ưu được ký hiệu  $\chi(G)$ . **Tập độc lập lớn nhất** là một tập đỉnh lớn nhất mà không có hai đỉnh nào kề nhau. **Phủ bằng clique nhỏ nhất** là phủ tất cả các đỉnh bằng số clique ít nhất.

Với mọi đồ thị,

$$\omega(G) \leq \chi(G),$$

vì các đỉnh trong cùng một clique phải nhận các màu khác nhau. Tương tự, kích thước tập độc lập lớn nhất không vượt quá số clique tối thiểu cần dùng để phủ các đỉnh.

## 2 Đồ thị dây cung

Một **dây cung** của một chu trình là cạnh nối hai đỉnh không liên tiếp trên chu trình. Một đồ thị vô hướng được gọi là **đồ thị dây cung** (chordal graph) nếu mọi chu trình độ dài lớn hơn 3 đều có ít nhất một dây cung. Nói tương đương, đồ thị không chứa chu trình cảm sinh  $C_n$  với  $n > 3$ .

Mọi đồ thị con cảm sinh của một đồ thị dây cung vẫn là đồ thị dây cung. Đây là tính chất quan trọng vì nhiều thuật toán dưới đây làm việc bằng cách xóa dần đỉnh và xét phần đồ thị còn lại.

### 2.1 Điểm đơn thuần

Ký hiệu  $N(v)$  là tập các đỉnh kề với  $v$ . Một đỉnh  $v$  là **đơn thuần** nếu đồ thị cảm sinh bởi  $\{v\} \cup N(v)$  là một clique.

**Bổ đề.** Mọi đồ thị dây cung đều có ít nhất một điểm đơn thuần. Nếu đồ thị dây cung không phải đồ thị đầy đủ, nó có ít nhất hai điểm đơn thuần không kề nhau.

Bổ đề này là nền tảng cho cách nhận biết đồ thị dây cung và xây dựng thứ tự khử hoàn hảo.

## 2.2 Thứ tự khử hoàn hảo

Một **thứ tự khử hoàn hảo** (perfect elimination ordering, PEO) là một hoán vị

$$v_1, v_2, \dots, v_n$$

của các đỉnh sao cho với mọi  $i$ , đỉnh  $v_i$  là điểm đơn thuần trong đồ thị cảm sinh bởi

$$\{v_i, v_{i+1}, \dots, v_n\}.$$

Nói cách khác, các láng giềng của  $v_i$  xuất hiện sau  $v_i$  trong thứ tự phải tạo thành một clique.

**Định lý.** Một đồ thị vô hướng là đồ thị dây cung khi và chỉ khi nó có một PEO.

**Chứng minh chiều đồ thị dây cung  $\Rightarrow$  PEO.** Vì mọi đồ thị con cảm sinh của đồ thị dây cung vẫn là dây cung, ta dùng quy nạp theo số đỉnh. Chọn một điểm đơn thuần, đặt nó ở đầu thứ tự khử, xóa nó, rồi áp dụng giả thiết quy nạp cho đồ thị còn lại.

**Chứng minh chiều PEO  $\Rightarrow$  đồ thị dây cung.** Giả sử ngược lại có một chu trình không dây cung độ dài lớn hơn 3. Lấy đỉnh  $v$  trên chu trình xuất hiện sớm nhất trong PEO. Hai láng giềng của  $v$  trên chu trình đều xuất hiện sau  $v$ , nên theo tính chất PEO chúng phải kề nhau. Cạnh này là một dây cung, mâu thuẫn.

## 3 Nhận biết đồ thị dây cung

Bài ví dụ trên slide là ZOI 1015 *Fishing Net*: cho một đồ thị vô hướng, hãy quyết định nó có phải đồ thị dây cung hay không.

Cách ngây thơ là lặp lại:

1. tìm một điểm đơn thuần  $v$ ;
2. đưa  $v$  vào PEO;
3. xóa  $v$  và các cạnh liên quan khỏi đồ thị.

Nếu xóa hết đỉnh thì đồ thị là dây cung; nếu tại một bước không còn điểm đơn thuần thì đồ thị không phải dây cung. Cách này có thể tới  $\mathcal{O}(n^4)$ . Các slide sau giới thiệu hai thuật toán tuyến tính để sinh thứ tự ứng viên: LexBFS và MCS. Sau đó ta kiểm tra thứ tự ứng viên trong  $\mathcal{O}(n + m)$ .

### 3.1 LexBFS

**LexBFS** là tìm kiếm theo chiều rộng theo thứ tự từ điển (lexicographic BFS). Thuật toán đánh số các đỉnh theo thứ tự từ  $n$  về 1. Với mỗi đỉnh chưa được đánh số, duy trì một danh sách gồm nhãn của các láng giềng đã được đánh số, sắp theo thứ tự giảm dần. Mỗi bước chọn đỉnh chưa được đánh số có danh sách lớn nhất theo thứ tự từ điển.

```

for each vertex  $v$ :
     $L(v) \leftarrow$  empty list
for  $i = n$  downto 1:
    choose an unnumbered vertex  $v_i$  with lexicographically largest  $L(v_i)$ 
    for each unnumbered neighbor  $w$  of  $v_i$ :
        append  $i$  to  $L(w)$ 

```

Khác với BFS thường, LexBFS mở rộng đỉnh theo một thứ tự phụ rất chặt. Để cài đặt trong  $\mathcal{O}(n + m)$ , ta chia các đỉnh chưa chọn vào các **xô**: các đỉnh cùng danh sách nằm trong một xô, còn các xô được sắp theo thứ tự giảm dần của danh sách.

1. Mỗi bước lấy một đỉnh từ xô đầu tiên.
2. Khi một đỉnh  $v$  được đánh số, mọi láng giềng chưa đánh số của  $v$  được chuyển sang xô mới tương ứng với danh sách sau khi thêm nhãn của  $v$ .
3. Mỗi lần cập nhật một đỉnh tạo nhiều nhất một xô mới; tại mọi thời điểm số xô không vượt quá  $n$ .

Do mỗi cạnh chỉ gây ra số thao tác hằng, tổng thời gian là  $\mathcal{O}(n + m)$ .

### 3.2 MCS

**MCS** (maximum cardinality search) cũng đánh số từ  $n$  về 1. Với mỗi đỉnh  $i$ , đặt  $\text{label}[i]$  là số láng giềng của  $i$  đã được đánh số. Mỗi bước chọn một đỉnh chưa đánh số có nhãn lớn nhất, đánh số nó, rồi tăng nhãn của các láng giềng chưa đánh số.

```

maximum_cardinality_search( $G$ ,  $\sigma$ ):
    for each vertex  $v$  in  $V(G)$ :
         $\text{label}[v] \leftarrow 0$ 
    for  $i = n$  downto 1:
        choose an unnumbered vertex  $v$  with maximum  $\text{label}[v]$ 
         $\sigma(v) \leftarrow i$ 
        for each unnumbered neighbor  $w$  of  $v$ :
             $\text{label}[w] \leftarrow \text{label}[w] + 1$ 

```

Với đồ thị dây cung, thứ tự do MCS sinh ra là một PEO. Cài đặt bằng các xô theo giá trị nhãn 0, 1, 2, ... cũng đạt  $\mathcal{O}(n + m)$ .

### 3.3 Kiểm tra một thứ tự

Cho một thứ tự ứng viên  $v_1, \dots, v_n$ , cách kiểm tra ngây thơ là với mỗi  $v_i$ , kiểm tra tất cả các láng giềng của  $v_i$  trong  $\{v_{i+1}, \dots, v_n\}$  có tạo thành clique hay không.

Có thể kiểm tra nhanh hơn. Gọi các láng giềng xuất hiện sau  $v_i$  là

$$v_{j_1}, v_{j_2}, \dots, v_{j_k},$$

trong đó  $j_1 < j_2 < \dots < j_k$ . Chỉ cần kiểm tra  $v_{j_1}$  kề với  $v_{j_2}, \dots, v_{j_k}$ . Nếu điều kiện này đúng với mọi  $i$ , thứ tự là PEO. Việc kiểm tra có thể thực hiện trong  $\mathcal{O}(n + m)$ .

Vì vậy bài toán nhận biết đồ thị dây cung được giải trong thời gian tuyến tính:

$$\mathcal{O}(n + m).$$

## 4 Clique cực đại trong đồ thị dây cung

Giả sử  $p(v)$  là vị trí của  $v$  trong PEO. Đặt

$$N^+(v) = \{w \mid (v, w) \in E, p(w) > p(v)\}.$$

Mỗi clique cực đại của đồ thị dây cung đều có dạng

$$\{v\} \cup N^+(v)$$

với một đỉnh  $v$  nào đó. Thật vậy, trong một clique cực đại  $C$ , lấy đỉnh  $v \in C$  xuất hiện sớm nhất trong PEO. Mọi đỉnh còn lại của  $C$  đều thuộc  $N^+(v)$ , và vì  $C$  cực đại nên  $C = \{v\} \cup N^+(v)$ .

Hệ quả là một đồ thị dây cung có nhiều nhất  $n$  clique cực đại.

Không phải mọi tập  $\{v\} \cup N^+(v)$  đều cực đại. Slide mô tả một cách kiểm tra tuyến tính bằng đỉnh kế tiếp. Đặt  $\text{next}(v)$  là đỉnh đầu tiên trong  $N^+(v)$ , nếu tồn tại. Tập  $\{v\} \cup N^+(v)$  không cực đại nếu tồn tại một đỉnh  $w$  xuất hiện trước  $v$  sao cho clique của  $w$  chứa nó. Điều kiện có thể kiểm tra qua các quan hệ  $\text{next}(w) = v$  và so sánh kích thước:

$$|N^+(v)| + 1 \leq |N^+(w)|.$$

Từ đó có thể liệt kê toàn bộ clique cực đại trong  $\mathcal{O}(n + m)$ .

## 5 Tối ưu trên đồ thị dây cung

### 5.1 Tô màu

Bài ví dụ là HNOI 2008 *Magic Kingdom*. Cần tô màu các đỉnh sao cho hai đỉnh kề nhau có màu khác nhau và số màu là nhỏ nhất.

Trên đồ thị dây cung, sau khi có PEO, ta duyệt thứ tự từ sau ra trước. Với mỗi đỉnh, gán màu nhỏ nhất chưa được dùng bởi các láng giềng đã tô của nó. Do các láng giềng phía sau tạo thành clique, nếu thuật toán dùng  $T$  màu thì tồn tại một clique kích thước  $T$ . Vì luôn có  $\omega(G) \leq \chi(G)$ , suy ra

$$\omega(G) = \chi(G) = T.$$

Độ phức tạp là  $\mathcal{O}(n + m)$ .

### 5.2 Tập độc lập lớn nhất

Để tìm tập độc lập lớn nhất, duyệt PEO từ trước ra sau và **chọn nếu có thể**: chọn  $v_i$  nếu nó không kề với đỉnh nào đã chọn. Thuật toán tham lam này tối ưu trên đồ thị dây cung.

### 5.3 Phủ bằng clique nhỏ nhất

Giả sử tập độc lập lớn nhất thu được là

$$\{p_1, p_2, \dots, p_t\}.$$

Khi đó các tập

$$\{p_1\} \cup N^+(p_1), \{p_2\} \cup N^+(p_2), \dots, \{p_t\} \cup N^+(p_t)$$

tạo thành một phủ bằng clique. Vì mọi phủ bằng clique phải dùng ít nhất một clique cho mỗi đỉnh của một tập độc lập, ta có:

$$\alpha(G) = \theta(G),$$

trong đó  $\alpha(G)$  là kích thước tập độc lập lớn nhất, còn  $\theta(G)$  là số clique ít nhất trong một phủ clique.

## 6 Đồ thị hoàn hảo

Một đồ thị  $G$  là **hoàn hảo** nếu với mọi đồ thị con cảm sinh  $H$  của  $G$ ,

$$\omega(H) = \chi(H).$$

Slide gọi đồ thị **co-perfect** qua đồ thị bù; nội dung chính là định lý đồ thị hoàn hảo yếu: một đồ thị là hoàn hảo khi và chỉ khi đồ thị bù của nó cũng hoàn hảo.

Đồ thị dây cung là đồ thị hoàn hảo, vì mọi đồ thị con cảm sinh của nó vẫn là dây cung và có thể tô màu tối ưu bằng PEO.

## 7 Đồ thị khoảng

Cho một họ các khoảng trên trục số. Đồ thị giao của họ này có một đỉnh cho mỗi khoảng, và hai đỉnh có cạnh khi và chỉ khi hai khoảng tương ứng giao nhau. Một đồ thị là **đồ thị khoảng** nếu nó là đồ thị giao của một họ khoảng.

**Định lý.** Mọi đồ thị khoảng đều là đồ thị dây cung.

**Ý tưởng chứng minh.** Giả sử có một chu trình không dây cung độ dài lớn hơn 3,

$$v_0, v_1, \dots, v_{\ell-1}, v_\ell = v_0.$$

Gọi  $I_i$  là khoảng ứng với  $v_i$ . Vì  $I_i$  giao  $I_{i+1}$  nhưng không giao  $I_{i+2}$ , các điểm giao liên tiếp buộc phải dịch chuyển đơn điệu dọc theo trục. Khi quay lại  $I_0$ , điều này mâu thuẫn với việc  $I_0$  không giao  $I_2$ . Do đó không tồn tại chu trình cảm sinh dài hơn 3.

### 7.1 Hai bài toán cổ điển

**Chọn nhiều khoảng không giao nhau nhất.** Cho  $n$  khoảng, cần chọn nhiều khoảng nhất sao cho các khoảng được chọn đôi một không giao nhau. Đây là bài toán tập độc lập lớn nhất trên đồ thị khoảng. Sắp xếp các khoảng theo đầu phải tăng dần, rồi duyệt và chọn một khoảng nếu nó không giao với khoảng đã chọn gần nhất. Độ phức tạp là  $\mathcal{O}(n \log n)$ .

**Tetris.** Có  $n$  khối, mỗi khối cao 1, khối thứ  $i$  chiếm đoạn ngang  $[L_i, R_i]$ . Cần chọn thứ tự thả khối để chiều cao cuối cùng nhỏ nhất. Đây là bài toán tô màu nhỏ nhất trên đồ thị khoảng: hai khối xung đột nếu các đoạn  $[L_i, R_i]$  giao nhau. Sắp xếp theo đầu phải giảm dần và gán màu nhỏ nhất có thể; dùng heap hoặc cây đoạn để duy trì màu khả dụng. Độ phức tạp là  $\mathcal{O}(n \log n)$ .

Với một họ khoảng cho trước, sắp xếp các khoảng theo đầu phải tăng dần cho ta một PEO của đồ thị khoảng tương ứng.

## 8 Phân rã cây và clique tree

Một **phân rã cây** của đồ thị  $G = (V, E)$  là cặp  $(X, T)$ , trong đó

$$X = \{X_1, X_2, \dots, X_m\}$$

là các tập con của  $V$ , còn  $T$  là một cây có tập đỉnh là  $X$ . Nó thỏa ba điều kiện:

1.  $X_1 \cup X_2 \cup \dots \cup X_m = V$ ;
2. với mỗi cạnh  $(u, v) \in E$ , tồn tại  $X_i$  chứa cả  $u$  và  $v$ ;
3. với mỗi đỉnh  $v$ , các tập  $X_i$  chứa  $v$  tạo thành một cây con liên thông của  $T$ .

Một **clique tree** của  $G$  là cây mà các đỉnh là các clique cực đại của  $G$ , và với mỗi đỉnh gốc  $v$  của  $G$ , tập các clique cực đại chứa  $v$  tạo thành một cây con liên thông.

**Định lý.** Một đồ thị vô hướng  $G$  là đồ thị dây cung khi và chỉ khi nó có một clique tree. Clique tree là một dạng phân rã cây đặc biệt.

**Dựng clique tree cho đồ thị dây cung.** Trước hết liệt kê tất cả clique cực đại. Dựng một đồ thị phụ có mỗi clique cực đại là một đỉnh; giữa hai clique đặt cạnh có trọng số bằng kích thước giao của chúng. Một cây khung lớn nhất của đồ thị phụ là một clique tree.

**Đặc trưng đồ thị khoảng.** Một đồ thị là đồ thị khoảng khi và chỉ khi nó có một clique tree là một đường đi. Nói theo ngôn ngữ clique cực đại: có thể sắp các clique cực đại thành một dãy sao cho với mỗi đỉnh  $v$ , các clique chứa  $v$  xuất hiện liên tiếp trong dãy.

## 9 Nhận biết đồ thị khoảng

Quy trình nhận biết trên slide:

1. kiểm tra đồ thị có phải dây cung hay không trong  $\mathcal{O}(n + m)$ ;
2. nếu là dây cung, liệt kê tất cả clique cực đại trong  $\mathcal{O}(n + m)$ ;
3. kiểm tra có tồn tại một thứ tự liên tiếp của các clique cực đại hay không, tức với mỗi đỉnh, các clique chứa nó đứng liên tiếp.

Bước cuối là bài toán **tính chất các số 1 liên tiếp**. Lập ma trận nhị phân có hàng là các đỉnh, cột là các clique cực đại, và phần tử bằng 1 khi đỉnh thuộc clique tương ứng. Cần hoán vị các cột sao cho trong mỗi hàng, các số 1 tạo thành một đoạn liên tiếp.

### 9.1 PQ-tree

PQ-tree biểu diễn gọn tập các hoán vị còn có thể thỏa các ràng buộc liên tiếp. Lá là các phần tử cần hoán vị. Có hai loại nút trong:

- nút  $P$ : các con có thể được hoán vị tùy ý;
- nút  $Q$ : thứ tự các con cố định, nhưng được phép đảo ngược toàn bộ.

Với tập phần tử  $X$ , ban đầu mọi thứ tự đều có thể. Mỗi ràng buộc  $I$  yêu cầu các phần tử trong  $I$  phải đứng liên tiếp. Dùng PQ-tree, mỗi ràng buộc có thể được xử lý trong  $\mathcal{O}(|I|)$ . Trong bài nhận biết đồ thị khoảng, số clique cực đại không vượt quá  $n$ , và mỗi đỉnh thuộc không quá  $\deg(v)$  clique cực đại, nên tổng thời gian vẫn là  $\mathcal{O}(n + m)$ .

Ví dụ slide xét

$$X = \{A, B, C, D\}, \quad I_1 = \{A, B, C\}, \quad I_2 = \{A, D\}.$$

PQ-tree sau khi áp hai ràng buộc biểu diễn đúng các thứ tự trong đó  $A, B, C$  liên tiếp và  $A, D$  cũng liên tiếp.

Slide ghi chú rằng cài đặt PQ-tree khá phức tạp, đồng thời nhắc tới các cách nhận biết đồ thị khoảng khác, như các thuật toán của Hsu (1992), Hsu–Ma (1999), và bài *A New Test for Interval Graphs*.

## 10 Ghi chú cuối

Bài trình bày kết thúc bằng email liên hệ [cdq10131@gmail.com](mailto:cdq10131@gmail.com). Các slide phụ cuối cùng lặp lại một lời phàn nàn về việc một số bản chia sẻ tài liệu trên mạng bị đặt phí “điểm tài sản”: tác giả cho rằng tài liệu đã được tải lên để cộng đồng cùng dùng thì không nên bị thu thêm điểm, và mong muốn có bản chia sẻ miễn phí.